Lab 6: The Timer Unit

10/15/2026

Daniel Xu
ECE 3218L

1 Overview

2 Procedure

1. **Program 1: Rectangular wave.**

Table 1: Program 1 at 440 Hz ($pw = 36364$ clicks, prescaler 1).

	Expected		Measured	
Duty cycle	High	Low	High	Low
12.5%	284.1 μs	1.989 ms	[]	[]
25%	568.2 μs	1.705 ms	[]	[]
50%	1.136 ms	1.136 ms	[]	[]
75%	1.705 ms	568.2 μs	[]	[]
87.5%	1.989 ms	284.1 μs	[]	[]

2. **Program 2: Tones.**

Table 2: Tone periods at prescaler 1 ($pw = 16$ MHz$/f$).

Note	Frequency (Hz)	pw (clicks)
C4	261.63	61155
D4	293.66	54485
E4	329.63	48539
F4	349.23	45815
G4	392.00	40816
A4	440.00	36364
B4	493.88	32397
C5	523.25	30578
D5	587.33	27242
E5	659.25	24270

3. **Program 3: Input capture.**

Table 3: Program 3 input and output frequencies.

Input (Hz)	Expected output (Hz)	Measured output (Hz)
1000	500	[]
[]	[]	[]
[]	[]	[]

3 Discussion

3.1 Program 1: Accuracy and Limits

3.2 Program 2: Frequency Range

3.3 Program 3: Measuring a Period

4 Conclusion

A Full Code

Code 1: Program 1. Rectangular wave (main.asm).

```
; Lab 6, Program 1: rectangular wave on OC1A (PB1, Uno pin 9)
; Calls SETWAVE once with a 440 Hz period and the duty cycle in DUTY,
; then waits while the Timer1 compare ISR generates the wave.

.cseg
.org 0x0000
        jmp RESET           ; reset
.org OC1Aaddr
        jmp TIM1_COMPA      ; Timer1 compare match A

.org INT_VECTORS_SIZE       ; code starts after the vector table
.include "wave.inc"

.equ PW   = 36364           ; 440 Hz at prescaler 1
.equ DUTY = DC50            ; DC12, DC25, DC50, DC75 or DC87

RESET:
        ; point the stack at the top of SRAM
        ldi r16,high(RAMEND)
        out SPH,r16
        ldi r16,low(RAMEND)
        out SPL,r16

        ldi r24,low(PW)
        ldi r25,high(PW)
        ldi r23,DUTY
        ldi r22,CLK_1
        rcall SETWAVE

        ; the ISR does the rest
SPIN:
        rjmp SPIN
```

Code 2: SETWAVE subroutine and compare ISR, used by Programs 1 and 2 (wave.inc).

```
; wave.inc: rectangular wave on OC1A (PB1) using Timer1 output compare A
; Used by programs 1 and 2. The including program must put
;     jmp TIM1_COMPA
; at OC1Aaddr in its vector table.

; duty cycle codes for SETWAVE (r23)
; bits 1:0 = how far to shift pw right, bit 7 = invert the result
.equ DC12  = 0x03           ; 12.5% = pw >> 3
.equ DC25  = 0x02           ; 25%   = pw >> 2
.equ DC50  = 0x01           ; 50%   = pw >> 1
.equ DC75  = 0x82           ; 75%   = NOT 25%
.equ DC87  = 0x83           ; 87.5% = NOT 12.5%

; prescaler codes for SETWAVE (r22), the CS12:CS10 bits of TCCR1B
.equ CLK_1    = 1
.equ CLK_8    = 2
.equ CLK_64   = 3
.equ CLK_256  = 4
.equ CLK_1024 = 5

.dseg
HIGHCNT: .byte 2            ; OCR1A value for the high part (high width - 1)
LOWCNT:  .byte 2            ; OCR1A value for the low part (low width - 1)
.cseg

; SETWAVE: start a rectangular wave on OC1A
; r25:r24 = pw, period in timer clicks (at least 8)
; r23 = duty cycle code (DC12 ... DC87)
; r22 = prescaler code (CLK_1 ... CLK_1024)
; leaves r22-r25 unchanged and enables interrupts globally
SETWAVE:
        push r16
        push r17
        push r18
        push r19
        push r20
        push r21

        clr r16
        sts TCCR1B,r16      ; stop the timer while setting up

        ; part = pw >> shift
        movw r18,r24
        mov r20,r23
        andi r20,0x03       ; shift count
SW_SHIFT:
        lsr r19
        ror r18
        dec r20
        brne SW_SHIFT

        ; r19:r18 = high width = part, r17:r16 = low width = pw - part
        movw r16,r24
        sub r16,r18
        sbc r17,r19
        sbrs r23,7          ; inverted duty cycle?
        rjmp SW_SAVE
```

```
58          movw r20,r16        ; yes -> swap high and low
59          movw r16,r18
60          movw r18,r20
61  SW_SAVE:
62          ; in CTC mode each part lasts OCR1A + 1 clicks, so save width - 1
63          subi r18,1
64          sbci r19,0
65          subi r16,1
66          sbci r17,0
67          sts HIGHCNT,r18
68          sts HIGHCNT+1,r19
69          sts LOWCNT,r16
70          sts LOWCNT+1,r17
71
72          ; PB1 is the output, starting high
73          sbi DDRB,1
74          sbi PORTB,1
75
76          ; the first compare match ends the high part
77          sts OCR1AH,r19      ; high byte first
78          sts OCR1AL,r18
79          clr r20
80          sts TCNT1H,r20      ; count from 0
81          sts TCNT1L,r20
82
83          ; once COM1A is set, PB1 follows the OC1A bit, not PORTB,
84          ; so force OC1A high: set-on-match mode + force compare,
85          ; then switch to toggle mode (WGM11:10 = 0)
86          ldi r20,(1<<COM1A1)|(1<<COM1A0)
87          sts TCCR1A,r20
88          ldi r20,(1<<FOC1A)
89          sts TCCR1C,r20
90          ldi r20,(1<<COM1A0) ; toggle OC1A on compare match
91          sts TCCR1A,r20
92
93          ldi r20,(1<<OCF1A)  ; clear a stale compare flag
94          out TIFR1,r20
95          ldi r20,(1<<OCIE1A) ; interrupt on compare match A
96          sts TIMSK1,r20
97
98          ; build TCCR1B in a register: CTC mode + prescaler
99          ; writing it starts the timer, so do it last
100         ldi r20,(1<<WGM12)
101         or r20,r22
102         sts TCCR1B,r20
103         sei
104
105         pop r21
106         pop r20
107         pop r19
108         pop r18
109         pop r17
110         pop r16
111         ret
112
113 ; TIM1_COMPA: OC1A just toggled and TCNT1 was cleared
114 ; load OCR1A with the width of the part that just started
115 TIM1_COMPA:
116         push r16
```

```
117         in r16,SREG
118         push r16
119         push r17
120
121         sbis PINB,1          ; PB1 high -> low?
122         rjmp TC_LOW
123         lds r16,HIGHCNT      ; low -> high: time the high part
124         lds r17,HIGHCNT+1
125         rjmp TC_SET
126 TC_LOW:
127         lds r16,LOWCNT       ; high -> low: time the low part
128         lds r17,LOWCNT+1
129 TC_SET:
130         sts OCR1AH,r17       ; high byte first
131         sts OCR1AL,r16
132
133         pop r17
134         pop r16
135         out SREG,r16
136         pop r16
137         reti
```

Code 3: Program 2. Tones (tones.asm).

```
1  ; Lab 6, Program 2: play a scale on a piezo speaker on OC1A (PB1, Uno pin 9)
2  ; Copies the tone periods into SRAM, then for each one: start a 50% duty
3  ; wave with SETWAVE, wait 500 ms, stop the timer, wait 500 ms.
4
5  .DEF ZERO=R0
6
7  .equ NTONES = 10
8
9  .dseg
10 TONES:  .byte 2*NTONES       ; tone periods in timer clicks, low byte first
11
12 .cseg
13 .org 0x0000
14         jmp RESET            ; reset
15 .org OC1Aaddr
16         jmp TIM1_COMPA       ; Timer1 compare match A
17
18 .org INT_VECTORS_SIZE        ; code starts after the vector table
19 .include "wave.inc"
20
21 RESET:
22         ; point the stack at the top of SRAM
23         ldi r16,high(RAMEND)
24         out SPH,r16
25         ldi r16,low(RAMEND)
26         out SPL,r16
27         clr ZERO             ; keep a constant 0 in r0
28
29         ; copy the period table from flash into TONES
30         ldi ZL,low(TONETAB*2)
31         ldi ZH,high(TONETAB*2)
32         ldi XL,low(TONES)
33         ldi XH,high(TONES)
34         ldi r16,2*NTONES
```

```
COPY:
        lpm r17,Z+
        st X+,r17
        dec r16
        brne COPY

        ; play each tone
        ldi YL,low(TONES)
        ldi YH,high(TONES)
        ldi r20,NTONES      ; tones left
NEXTTONE:
        ld r24,Y+           ; pw
        ld r25,Y+
        ldi r23,DC50
        ldi r22,CLK_1
        rcall SETWAVE       ; tone on

        ldi r24,low(500)
        ldi r25,high(500)
        rcall waitnms

        sts TCCR1B,ZERO     ; clear the prescaler bits -> timer stops, tone off

        ldi r24,low(500)
        ldi r25,high(500)
        rcall waitnms

        dec r20
        brne NEXTTONE

DONE:
        rjmp DONE

; tone periods at prescaler 1: 16 MHz / f
TONETAB:
        .dw 61155           ; C4 261.63 Hz
        .dw 54485           ; D4 293.66 Hz
        .dw 48539           ; E4 329.63 Hz
        .dw 45815           ; F4 349.23 Hz
        .dw 40816           ; G4 392.00 Hz
        .dw 36364           ; A4 440.00 Hz
        .dw 32397           ; B4 493.88 Hz
        .dw 30578           ; C5 523.25 Hz
        .dw 27242           ; D5 587.33 Hz
        .dw 24270           ; E5 659.25 Hz

; 1 msec timer subroutine (exact; time includes call and return)
; wait1ms expends 16000 clock cycles.
; rcall wait1ms uses 3 clock cycles
; Therefore from the rcall in the calling program through the
; ret in the subroutine wait1ms expends
; 3+1+1+1+(3996*4)[in loop]+3[end of loop]+1+1+1+4 = 16000cc
; which at 16MHz (cc/s) equals 1ms.
wait1ms:
        ldi zh,0x0F         ; 1cc
        ldi zl,0x9D         ; 1cc
        clz                 ; 1cc
waitloop:                   ; countdown 0x0F9D = 3997dec
        sbiw z,1            ; 2cc
```

```
94          brne waitloop       ; 2cc true, 1cc false
95          nop                 ; 1cc
96          nop                 ; 1cc
97          nop                 ; 1cc
98          ret                 ; 4cc
99
100 ; waitnms: wait n ms
101 ; load n into r24, r25
102 ; r24, r25 and Z are restored before returning
103 ; the compare ISR runs during the wait and stretches it slightly
104 waitnms:
105         push r24            ; 2
106         push r25            ; 2
107         push zl             ; 2
108         push zh             ; 2
109         cp r24,ZERO         ; 1
110         cpc r25,ZERO        ; 1
111         breq waitnmsdone    ; 1 if n != 0
112 waitnmsloop:
113         rcall wait1ms       ; 16000 total with rcall and ret
114         sbiw r25:r24,1      ; 2
115         brne waitnmsloop    ; 2 if taken, 1 if not
116 waitnmsdone:
117         pop zh              ; 2
118         pop zl              ; 2
119         pop r25             ; 2
120         pop r24             ; 2
121         ret                 ; 4
```

Code 4: Program 3. Input capture (capture.asm).

```
1  ; Lab 6, Program 3: half-frequency copy of an input signal
2  ; Input capture on ICP1 (PB0, Uno pin 8) measures the input period.
3  ; Output compare toggles OC1A (PB1, Uno pin 9) once per input period,
4  ; so the output has twice the period (half the frequency) of the input.
5  ; Timer1 runs free (normal mode) at clk/8 = 2 MHz, 0.5 us per click,
6  ; so periods up to 65535 clicks = 32.77 ms (inputs above 30.5 Hz) work.
7
8  .dseg
9  PREVCAP: .byte 2            ; ICR1 at the previous edge
10 PERIOD:  .byte 2            ; input period in clicks = output half period
11 ROLLED:  .byte 1            ; 1 if the timer rolled over during the last period
12
13 .cseg
14 .org 0x0000
15         jmp RESET           ; reset
16 .org ICP1addr
17         jmp TIM1_CAPT       ; Timer1 input capture
18 .org OC1Aaddr
19         jmp TIM1_COMPA      ; Timer1 compare match A
20
21 .org INT_VECTORS_SIZE       ; code starts after the vector table
22 RESET:
23         ; point the stack at the top of SRAM
24         ldi r16,high(RAMEND)
25         out SPH,r16
26         ldi r16,low(RAMEND)
27         out SPL,r16
```

```
28
29          ; clear the variables
30          clr r16
31          sts PREVCAP,r16
32          sts PREVCAP+1,r16
33          sts PERIOD,r16
34          sts PERIOD+1,r16
35          sts ROLLED,r16
36
37          cbi DDRB,0          ; PB0 = ICP1 input
38          sbi DDRB,1          ; PB1 = OC1A output
39
40          ldi r16,(1<<COM1A0) ; toggle OC1A on compare match, normal mode
41          sts TCCR1A,r16
42
43          ldi r16,(1<<ICF1)|(1<<OCF1A)    ; clear stale flags
44          out TIFR1,r16
45          ldi r16,(1<<ICIE1)|(1<<OCIE1A)  ; capture and compare A interrupts
46          sts TIMSK1,r16
47
48          ; noise canceller, capture on rising edge, clk/8 (starts the timer)
49          ldi r16,(1<<ICNC1)|(1<<ICES1)|(1<<CS11)
50          sts TCCR1B,r16
51          sei
52
53          ; the ISRs do the rest
54  SPIN:
55          rjmp SPIN
56
57  ; TIM1_CAPT: a rising edge arrived on ICP1
58  ; PERIOD = ICR1 - previous ICR1
59  TIM1_CAPT:
60          push r16
61          in r16,SREG
62          push r16
63          push r17
64          push r18
65          push r19
66
67          lds r16,ICR1L       ; low byte first
68          lds r17,ICR1H
69          lds r18,PREVCAP
70          lds r19,PREVCAP+1
71          sts PREVCAP,r16     ; this edge is the next one's previous edge
72          sts PREVCAP+1,r17
73
74          ; now - prev; borrow (C = 1) means now < prev, so the timer rolled
75          ; over once. The 16-bit result already equals now + 65536 - prev,
76          ; which is the right period, so only the flag is recorded.
77          sub r16,r18
78          sbc r17,r19
79          clr r18             ; clr leaves C alone
80          rol r18             ; r18 = rollover flag
81          sts ROLLED,r18
82
83          sts PERIOD,r16
84          sts PERIOD+1,r17
85
86          pop r19
```

```
87          pop r18
88          pop r17
89          pop r16
90          out SREG,r16
91          pop r16
92          reti
93
94  ; TIM1_COMPA: OC1A just toggled; schedule the next toggle
95  ; OCR1A = OCR1A + PERIOD. Adding to OCR1A instead of TCNT1 keeps the
96  ; ISR's own latency from adding to every half period.
97  TIM1_COMPA:
98          push r16
99          in r16,SREG
100         push r16
101         push r17
102         push r18
103         push r19
104
105         lds r16,OCR1AL
106         lds r17,OCR1AH
107         lds r18,PERIOD
108         lds r19,PERIOD+1
109         add r16,r18
110         adc r17,r19
111         sts OCR1AH,r17      ; high byte first
112         sts OCR1AL,r16
113
114         pop r19
115         pop r18
116         pop r17
117         pop r16
118         out SREG,r16
119         pop r16
120         reti
```